Imitation Learning for Vision-Based Object Collection in MiniWorld

Tommaso Calzolari Leonardo Pedretti

Abstract

This project addresses a vision-based object-collection task using imitation learning. A robot operates in a MiniWorld environment and receives only a low-resolution RGB image from its current viewpoint. At every timestep, the policy must choose one of three actions: turn left, turn right, or move forward. The task is to collect yellow duck objects in several visually different rooms. We manually played the game to obtain expert demonstrations, converted the demonstrations into supervised image-action pairs, designed an interpretable feature extractor for the yellow targets, compared classical and neural classifiers, and trained a compact multilayer perceptron policy. Rooms 0, 1, and 2 were used for training and validation, while room 3 was kept as an unseen test environment. In a 50-run closed-loop evaluation on room 3, the policy collected an average of 4.30 out of 5 objects per episode, with an average success rate of 0.86.

1 Problem Formulation

The task can be formulated as learning a policy:

$$\pi(o_t) \to a_t, \tag{1}$$

where $o_t \in \mathbb{R}^{60\times80\times3}$ is the RGB camera observation at time t, and $a_t \in \{0, 1, 2\}$ is the selected action. The action labels are: 0 for turn left, 1 for turn right, and 2 for move forward.

The simulator gives a positive reward when the robot reaches and picks up a yellow object. Rather than using reinforcement learning, we treated the task as behavior cloning: human demonstrations provide pairs (o_i, a_i), and a supervised classifier learns to imitate the demonstrator.

2 Feature Engineering

The feature extractor is based on simple image-processing operations. For each RGB observation, we first construct a binary yellow mask:

$$M(x,y) = \begin{cases} 1, & \text{if } R(x,y) \geq 200,\ G(x,y) \geq 200,\ B(x,y) \leq 50, \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

After thresholding, connected-component labeling is applied to the binary mask. If multiple yellow blobs are visible, only the largest connected component is retained, since it is the most likely current target. From this component and from the full mask, we compute four features:

$$x_c = \frac{1}{|\Omega|} \sum_{(x,y)\in\Omega} x, \qquad y_c = \frac{1}{|\Omega|} \sum_{(x,y)\in\Omega} y, \tag{3}$$

where Ω is the largest yellow component. We also compute the yellow-pixel fraction

$$q = \frac{\sum_{x,y} M(x,y)}{HW}, \tag{4}$$

and a left-right skew feature

$$s = \frac{N_{\text{left}} - N_{\text{right}}}{N_{\text{yellow}} + \epsilon}. \tag{5}$$

The feature vector is

$$\phi(o) = [x_c, y_c, s, q]. \tag{6}$$

3 Hyperparameter Tuning

The most important hyperparameters were the number of hidden units, number of hidden layers, dropout rate, L2 regularization strength, learning rate, batch size, number of epochs, and early-stopping patience.

Configuration	h_1	h_2	Dropout	L2	Learning rate	Batch size
Selected MLP	32	16	0.0	10^{-4}	10^{-3}	128

Table 1: MLP hyperparameters selected after random search.

The model uses batch normalization, two ReLU hidden layers with 32 and 16 units, and a 3-way softmax output. Training uses Adam, sparse categorical cross-entropy, early stopping, and softened class weights.

4 Policy Deployment

The policy loads the saved Keras model `best_model.keras`, extracts the four image features from each observation, predicts the action probabilities, and returns the action with maximum probability. The policy is stateless: it uses only the current frame. Despite this limitation, the engineered features encode the control-relevant information needed for target alignment and approach.

5 Evaluation

The model was evaluated in closed loop on room 3, the unseen test room. We ran 50 deterministic seeds. Each run contains 5 target objects, so a success rate of 1.0 means that all 5 objects were collected.

Metric	Result on room 3
Number of runs	50
Average success rate	0.860
Average objects collected	4.30 / 5
Median objects collected	5 / 5
Full-success runs	30 / 50
Runs with at least 4 objects	41 / 50
Average reward	0.02195

Table 2: Closed-loop evaluation of the policy on the unseen room 3 test environment.

The object-count distribution over the 50 test runs was:

$$\{1{:}2,\ 2{:}2,\ 3{:}5,\ 4{:}11,\ 5{:}30\}. \tag{7}$$

Thus, the majority of runs collected all objects, and 82% of runs collected at least 4 out of 5 objects.

6 Conclusion

The project demonstrates a complete imitation-learning pipeline for a vision-based robotics task: demonstration collection, data inspection, feature engineering, model comparison, hyperparameter tuning, cross-room validation, and unseen-room deployment. The MLP policy is compact, interpretable, and effective in closed loop. On 50 unseen room-3 test runs, it achieved an average success rate of 0.86 and collected a median of 5 out of 5 objects, showing that the manually engineered visual features generalize well across room appearance changes.